

version: 1.16
date: 2026-09-06
status: companion-seventh-closing-pass-applied
author: BOSUN (drafting); Cathal Ryan Hynes (principal)
companion_to: ZeeBeam manuscript v3.20 (paper/zeebeam.md)

ZeeBeam, by Example

From a toy proof to the ZeeBeam relations, one step at a time

This is the worked-examples companion to the manuscript *ZeeBeam: The Zero-Knowledge Beam* (v3.20). ZeeBeam names the capture-and-proof system, and a ZeeBeam recording is a session with rows proved under it; the work belongs to Dark Lantern, the wider privacy and zero-knowledge research programme. The manuscript states the relation, the proofs and the limits in the compressed form a referee expects. This document climbs to the same place slowly, with small examples whose numbers you can check by hand or with three lines of Python, and it says at each step how and why the ZeeBeam construction uses the idea. Every figure quoted for the real system is taken from the manuscript and the artifact bundle; nothing here claims more than they do. Where the manuscript says a thing is conditional or unproved, so does this.

A reader who wants only the picture can read Sections 1, 5, 8 and 9. A reader who wants to run the verifier needs Section 10.

1. What a proof is here

Every proof in this story has the same shape. There is a public statement x that everyone can see. There is a private witness w that only the prover holds. There is a relation R , a fixed, published check, such that $R(x, w)$ is either true or false. The prover wants to convince a verifier that they know a w making $R(x, w)$ true, and the verifier wants to be convinced without being shown w .

Three properties make such a proof worth having. **Completeness:** an honest prover with a valid w always convinces the verifier. **Soundness:** a prover without a valid w cannot convince the verifier except with negligible probability, however clever they are. **Zero knowledge:** the proof reveals no more than the public statement, because

everything the verifier sees could have been produced from the statement alone. The last property is what the letters ZK stand for.

One caution about how the manuscript uses these words. Its theorem is about soundness: under stated assumptions, an accepting proof means the program's checks were satisfied (execution binding, Section 9.1). It separately records that the 1,101-byte statement contains no pixels of the frame (Section 9.7). SP1, the proof system used, is designed as a zero-knowledge system, but the manuscript does not state or prove a privacy theorem for this particular artifact, so this document does not claim one either. What it says is exactly this: the verifier is handed a statement with no pixels in it and a proof that the checks passed.

Two more words recur. A **commitment** is a short public value that pins a private value: once published, the private value cannot be changed without detection. A **hash** is the workhorse of commitments in this construction, with one caveat the next section explains.

2. Example one: a hash and a hidden preimage

SHA-256 turns any input into 32 bytes. Two real values, computed on the ship's machine:

```
SHA-256("CittaDel") = be8b37e7031838cd072205ce6ebba28b6275e13dc5054e9610be2fbab49f1174
SHA-256("cittaDel") = a3758122e0a4f76121d3a31d7bac99a633ffae69966e39114a5955302ecbedcd
```

One changed letter and every hex digit moves. Nobody knows how to find two different inputs with the same SHA-256 (collision resistance; manuscript assumption A3), and nobody knows how to work backwards from the output to an unknown input (preimage resistance). So publishing `be8b37e7...1174` **binds** me to the string "CittaDel": if I later claim a different string, anyone can hash it and catch me.

The caveat: a plain hash supplies computational binding (the manuscript's A3) but no hiding guarantee. Anyone can hash candidate strings, and "CittaDel" is a very guessable candidate on this barge; byte length alone does not establish that a frame is unguessable. The manuscript's claim is narrower: the public statement contains no pixels, while the digest binds the relation to particular bytes; Section 1 states it that way.

The first zero-knowledge-style statement is: **"I know a string whose SHA-256 is be8b37e7...1174."** Opening the commitment would prove it but would reveal the string. A

proof of the statement shows it without opening. In the ZeeBeam relation the same idea appears at scale: the raw frame's BLAKE3 digest (a hash used here under the same collision-resistance assumption A3; for a feel of it, `BLAKE3("hello") = ea8f163d...200f`) is folded into the session's chain, and Theorem 1 binds the published outputs of the pinned functions to those exact frame bytes without placing the pixels in the statement.

Why it matters for ZeeBeam. Every object the proof reasons about, the frame, the projected pattern, the chain state, the 712-row session, the Zcash transaction, is pinned by a digest. The proof never handles "a frame"; it handles "the frame whose BLAKE3 is the one in the chain".

3. Example two: a proof that reveals nothing, with numbers small enough to follow

Take the sixteen nonzero numbers modulo 17 and multiply them modulo 17. They form a group of sixteen elements, and the number 3 generates all of them:

```
3^1..3^16 mod 17 = 3, 9, 10, 13, 5, 15, 11, 16, 14, 8, 7, 4, 12, 2, 6, 1
```

Public statement: $y = 7$. Private witness: $x = 11$, because $3^{11} \bmod 17 = 7$. Finding x from y is the discrete-logarithm problem; in this sixteen-element group it is trivial, in a prime-order group of about 2^{255} elements, the size used in real systems, it is believed infeasible.

Here is the classic interactive proof (Schnorr's protocol) that the prover knows x , run once with real numbers and a one-bit challenge.

1. The prover picks a random k , say $k = 5$, and sends $t = 3^5 \bmod 17 = 5$.
2. The verifier sends a random challenge c from $\{0, 1\}$, say $c = 1$.
3. The prover answers $s = k + c \cdot x \bmod 16 = 5 + 11 \bmod 16 = 0$.
4. The verifier checks $3^s \equiv t \cdot y^c \pmod{17}$: $3^0 = 1$, and $5 \cdot 7 = 35 \equiv 1 \pmod{17}$. Accepted.

Had the challenge been $c = 0$, the answer would have been $s = 5$ and the check $3^5 = 5 = t$. Accepted again. A prover who can answer both challenges for the same t knows x : subtracting the two answers gives $x = s_1 - s_0 \bmod 16$. A cheater who prepared for only one challenge is caught with probability $1/2$ per run; after eighty independent runs the chance of guessing every challenge is 2^{-80} , about 8.3×10^{-25} , below one in 10^{24} .

Why the challenge is only one bit in this toy: sixteen is not a prime, and in a group of composite order the bookkeeping breaks. With sixteen challenges a cheater could send $t = 1$ and answer both $c = 0$ ($s = 0$) and $c = 8$ ($s = 8$), since $3^8 \equiv 7^8 \equiv 16 \pmod{17}$, without knowing x . Real systems use prime-order groups and challenges of about 254 bits, where one run suffices.

Why does an honest verifier's transcript reveal nothing beyond y ? Because a transcript (t, c, s) with the same distribution can be produced without x : pick c and s first, then set $t = 3^s \cdot y^{-c}$, where y^{-c} is the modular inverse. The verifier's view is simulable, so it carries no information about x . This is an honest-verifier illustration on a toy group, not a security result.

The last trick makes the proof a single message. Instead of a live verifier choosing c , derive it by hashing the transcript so far, $c = \text{hash}(t, y)$, and mapping the digest into the challenge space. This is the Fiat-Shamir transform, and it is how every proof in the ZeeBeam bundle is a file rather than a conversation. It is not free: a prover can try many commitments t looking for a lucky challenge, so its security is a quantified soundness error rather than an impossibility, and manuscript assumption A1 names that error, together with the Groth16 setup assumption, as part of what Theorem 1 rests on (A1, A2 and A3 together).

Why it matters for ZeeBeam. ZeeBeam does not run Schnorr. Two of its legs are algebraic checks of the same family, public group elements, private scalars and an equation the verifier can test: the beacon leg verifies a BLS pairing equation over BLS12-381 (Section 5.2 "beacon"), and the chameleon and trapdoor legs verify Baby Jubjub group equations (Section 5.2 "chameleon", "trapdoor"). The reader needs only the shape.

4. Example three: proving membership in a tree

Take four leaves and hash them; then hash the pairs; then hash the two results. Real values (`||` joins the raw 32-byte digests, not their hexadecimal text):

```
leaf0 = SHA-256("row 0: frame A") = 1cfebe87c97b79c0...
leaf1 = SHA-256("row 1: frame B") = 449a41e3abc65628...
leaf2 = SHA-256("row 2: frame C") = 8ae21941fb7905e1...
leaf3 = SHA-256("row 3: frame D") = 892f356afec72b4d...
node01 = SHA-256(leaf0 || leaf1) = ae8c4625d4b859ec...
node23 = SHA-256(leaf2 || leaf3) = 334419e44623e804...
root   = SHA-256(node01 || node23) = 627972237f0bc016...
```

The root is a commitment to all four leaves at once. To prove that "row 2: frame C" is in the tree, show the leaf (or its digest) with its position, and its two siblings, leaf3 and node01; the verifier recomputes node23 and then the root and compares. The authentication path is two sibling hashes for four leaves, ten for a thousand leaves. This is a Merkle tree.

Shown in the clear, that is a proof of membership but not a zero-knowledge one: the verifier sees the leaf. Run the same recomputation inside a proof and the verifier learns only that some leaf with a stated property sits under the published root.

Why it matters for ZeeBeam. The 712 rows of the session form a BLAKE3 Merkle tree of depth 10 (1,024 slots). The remaining 312 slots hold domain-separated padding leaves; passing one off as a row would require a BLAKE3 collision (A3). Each row leaf hashes the session context, the row index, the chain state before and after (S_t and S_{t+1}), the frame's digest, the 28-byte capture metadata, the beacon round r_t and value v_t , and the pattern's digest (manuscript Section 4). The root is wrapped with a session context digest that includes the row count, the depth, the session identifier, S_0 , S_N and the authority-manifest and chain-log digests, so a proof for a row also names the session it belongs to. The per-row proof opens one leaf against the published root with ten siblings, in 121,601 instructions, one of the cheapest legs (Section 6). The whole-session chain proof of Section 9.5 rebuilds this tree from all 712 rows and publishes the same root, which is how the two proofs are known to speak of one tree.

5. Example four: proving that a program ran

One route to a proof of a relation is to hand-build an arithmetic circuit for it. The ZeeBeam relation instead uses a **zkVM**, SP1 6.4.0: an ordinary program, written in Rust and compiled to a RISC-V binary (the "guest ELF"), is executed by a prover that emits a proof of the statement "this program, run on some private input, wrote exactly these public bytes".

Three consequences shape everything that follows.

The program is identified by a key. SP1 derives a 32-byte verification-key hash from the compiled ELF: `0x0019d16cfe911d943315c04235c792fcccc3e0759aa3095a941b8508178e0490` for the final per-row relation. A verifier who pins that key is pinning the exact program that was proved. Manuscript assumption A2 adds what the key cannot carry by itself: the verifier must obtain it through a trusted channel and trust its connection to the audited source, the pinned toolchain and the lockfile. A change that alters the ELF

normally changes the key, and Section 7.3 records how sharp that is: a five-line replacement of a three-line comment, with no code change, produced a different ELF and a different key, because Rust compiles the line number of every `expect`, `unwrap` and `index` into the binary for its panic messages. The frozen source tree is therefore kept byte for byte, and the release ELF is built with machine paths remapped so that two machines produce the same bytes. The ceremony-2 box and, later, all eight fleet boxes rebuilt the guest and matched the pinned digest and key before proving.

Cost is counted in instructions. The per-row program executes about 4.15 billion RISC-V instructions (row 96: 4,149,712,293). Proving that on one A100 GPU took 12 min 08 s to 15 min 56 s per row across the 257 fleet rows, median 12 min 36 s, and 12 min 30 s and 14 min 17 s for the two ceremony rows (Section 7.1). Executing it without proving takes about a minute on the development machine. The proof is not a faster way to run the program; it is a way to let anyone holding the artifact and the trusted key hash check one run, under the stated assumptions and the out-of-band checks of Section 10.

The final proof is small and verification is cheap. SP1 produces a large STARK proof, compresses it recursively, and wraps the result in a Groth16 proof on the BN254 curve. The raw Groth16 proof in the bundle is 356 bytes (a 4-byte key-hash prefix and the encoded proof), whatever the program's size. The public statement is 1,101 bytes. The standalone verifier in the bundle does its cryptography with the `sp1-verifier 6.4.0` crate and uses `sha2` to print digests; its locked dependency graph has 209 packages. The bundle's record `verifier/results/halo_timing_20260903.txt` gives the development machine's measurement: one invocation, which verifies the proof, then flips one statement byte and expects rejection, then tries a wrong key and expects rejection, took 1.64 s of wall clock and 2.3 MB of memory, with a 917,512-byte binary. Proving the median row cost about \$0.42 of GPU time at the fleet's list price, for the prove step alone; the recorded standalone verification of one proof with both tamper controls took 1.64 s on the development machine (`verifier/results/halo_timing_20260903.txt`).

Why it matters for ZeeBeam. The relation of Section 5.2 is not a handful of algebraic constraints. It is BLAKE3 over 24.5 megabytes, a fixed-point image renderer, two integer neural networks, BLS pairings, Zcash transaction parsing and note decryption, all in one program. A zkVM makes that a matter of writing careful Rust rather than designing circuits, and lets an auditor read the checks as source code (`source/rust/.../program/src/main.rs` in the bundle). Other proof systems could encode the same relation differently; this is the route the project took.

6. Example five: the relation, leg by leg

Now the real thing. The public statement is 1,101 bytes laid out in the table of manuscript Section 5.1. The private witness consists of the thirteen inputs the guest reads: the row header, the membership witness, the raw frame, the beacon signature σ_t , the anchor transaction bytes, the anchor Merkle branch, the anchor block header, the incoming viewing key, the chain-log prefix, PREFIX.json, the receipt, the opening and the trapdoor.

Leg A. The beacon (4,904,590 instructions). drand's quicknet publishes a BLS signature every three seconds; the signature on round r is a point on the BLS12-381 curve, and the round's random value is its SHA-256. The program verifies the signature for the row's round r_t under the quicknet public key, the pairing equation $e(\sigma, G2) = e(H(r), pk)$, and derives the value $v_t = \text{SHA-256}(\sigma)$. This is the algebraic-check idea of Section 3 in its pairing form, verified inside the proof rather than trusted from a website. The round number is published in the statement; the signature need not be, because the proof already verified it. SP1 6.4.0 has no pairing syscall, so the patched `bls12_381` crate performs the pairing in software over accelerated field arithmetic; the whole check costs 0.1% of the program.

Leg B. The chain advance (702,715,828 for the frame hash). The session runs a hash chain. State S_{t+1} is BLAKE3 over a domain tag, the previous state S_t , the BLAKE3 of the raw frame, 28 bytes of capture metadata, the beacon round and the beacon value. The program hashes the entire 24,472,000-byte frame and recomputes S_{t+1} . This is Example one at scale: the frame is committed by its digest, and the digest is folded into a state that every later row depends on.

Leg C. The previous row (6,733,568). The nominal pattern against which row t is checked derives from S_t , which was formed at row $t-1$ from round r_{t-1} 's value. So the program also reads row $t-1$ from the anchored chain-log prefix, verifies that row's beacon signature too, and recomputes S_t . Both rounds are published. This leg exists because a referee found, within the first hour of the first round, that verifying only r_t bounded the wrong frame's pattern (Section 7.4). Row 0 has no predecessor and is refused. So the per-row relation verifies two transitions, into S_t and into S_{t+1} , and none of the session's other transitions; those are the chain proof's job (Section 8).

Leg D. The emission render (2,141,120,868, 51.6%; the XOF expansion is a separate 4,829,169). From S_t the program derives three seeds, expands each with BLAKE3's extendable output into 43,110 bytes, and renders a $1,920 \times 1,080 \times 3$ pattern with a four-octave fixed-point bilinear interpolation. Every step is BLAKE3 or integer arithmetic; there is no floating point in the guest. This is the most expensive

leg by far, and it is what makes the pattern a deterministic function of the verified predecessor-round value. What that means for time is conditional: under A4 the signature is computationally unavailable before its scheduled release (not nonexistent; the signing key determines it at all times), and under premise A8, which the manuscript states but does not prove, so is the pattern.

Leg E. Preprocess and typed root (421,045,347 and 14,810,611). The Bayer frame is packed once into four colour planes; a 1,024-pixel-square window of the camera planes and a matching window of the pattern are block-averaged to 256×256 ; a 32-leaf BLAKE3 tile tree over the pair gives a typed context and root for the camera-pattern pair the coupling network scores. A development incident lives here: the raw frame and its packed planes have exactly the same byte length, and an early version scored the wrong one. The circuit did not notice. The oracle of Section 7 did.

Leg F. The coupling score (338,997,588). A frozen integer neural network (trained-r32, post-training quantised) scores whether the camera window and the pattern window belong together. Its output is an integer numerator over a denominator of 4, published in the statement. The network's weights are embedded in the program and their SHA-256 is compared against a compiled constant, so the statement's "the frozen network said X" is exactly that.

Leg G. The pose verdict (344,916,129). A second frozen integer network (uncr64) works on a separate input: the whole frame reduced to four 256×256 planes, committed as the "uncropped commitment" published in the statement, then averaged in non-overlapping 4×4 blocks to four 64×64 planes. It produces eleven logit sums, a first-maximum verdict and a saturation count, all published. The manuscript is careful here (Section 8): the verdict is the output of a fixed function and nothing more; it agrees with the cue annotation on 101 of 116 evaluation rows, and on this single take the pose class is confounded with time of capture.

Leg H. Membership (121,601). The row's leaf, built from the values the program itself computed, is opened against the published session root with ten siblings, exactly as in Example three.

Leg I. Zcash inclusion (235,697). The program parses a version-6 (Ironwood) Zcash transaction from raw bytes, recomputes its ZIP-244 identifier from five digests, walks a Merkle branch of 33-byte levels to a block's Merkle root, and hashes the supplied block header twice with SHA-256. The published txid is 8d1672...f206. According to the project records, the supplied header is that of Zcash block 3456294 and the transaction was mined on 22 August 2026 minutes after the rows named by the supplied prefix; neither the proof nor the receipt establishes that chronology. What the proof cannot do is say which chain that header belongs to: the verifier must confirm,

out of band, that it is canonical Zcash mainnet block 3456294 and that the transaction is in it (Section 10).

Leg J. The memo (15,394,564). The transaction's first shielded action carries an encrypted memo. The program runs the Orchard library's `try_note_decryption` under the operator's 64-byte incoming viewing key, recomputes the note commitment and checks it against the action's, regenerates the ephemeral public key from the derived key, and reads the 64 hex digits after `binding=` in the plaintext. That binding is the digest of the receipt of the next leg. The manuscript's argument (Section 9.3, assumption A5) is that, for the pinned transaction and action, every viewing key that makes this leg accept yields the same binding value except with negligible probability, because the extracted note commitment is binding and decryption under a fixed key is deterministic. The AEAD cipher is expressly not assumed to be key-committing; the argument routes through the commitment instead.

Leg K. Prefix, receipt, commitment, trapdoor (126,039,564, all together).

Define the objects first. `PREFIX.json` is a 603-byte record naming the BLAKE3 of the 134,188-byte serialised chain-log prefix covering rows 0 to 259. The receipt is an 853-byte record whose SHA-256 is the memo's binding. It carries a Baby Jubjub point Y and a commitment $C = m \cdot \text{Base8} + r \cdot Y$, where `Base8` is a fixed generator compiled into the program, m is a scalar derived by SHA-512 from the receipt's canonical content (including the `contentRoot` computed from `PREFIX.json`), and r is the opening. The trapdoor td is a scalar with $Y = td \cdot \text{Base8}$; whoever holds it can open C to any message. The program hashes the prefix and checks that the proved row's chain state, frame digest, round and value appear inside it unchanged; computes the `contentRoot`; checks that SHA-256 of the receipt equals the decrypted binding; parses Y and C from canonical decimals, validates both on the curve and in the prime-order subgroup; recomputes C ; and checks whether the supplied td generates Y . The trapdoor flag in the last statement byte is 1 in every per-row proof in this bundle. Section 9.4 says exactly what this establishes: the transaction's action-0 ciphertext decrypts to a memo binding equal to SHA-256 of the receipt, and the proof demonstrates an opening from the receipt to this prefix. Because the operator knew td , C binds nothing against the operator and gives no upper bound on when this prefix existed; the anchor is an inclusion receipt, not a timestamp. A plain hash commitment next time would remove the caveat.

The legs interlock (Section 5.3). The beacon value enters the chain state; the chain state produces the pattern; the frame's digest enters the next state and the leaf; the leaf opens into the root; the row's values sit inside the anchored prefix; the prefix is named by the receipt; the receipt by the memo; the memo by the transaction; the transaction by the header. Under A1 to A3, a witness that alters any in-circuit value

must still satisfy every checked equality, so an accepting proof should not exist except with the proof system's soundness error. Whether the header belongs to the canonical chain is not one of those in-circuit links; it stays an external check.

7. Example six: the statement, and how it is checked against something other than itself

A proof shows that the program wrote the statement. It does not show that the program computes what its authors think it computes. The ZeeBeam work handles that with **oracle discipline** (Section 6): for every proved row, the host builds the expected 1,101-byte statement without the circuit, from the records and from Python re-runs of the two frozen networks reconstructed from their blobs, and compares byte for byte. For all 259 anchored rows every statement byte matched an oracle value and none rested on the circuit alone.

The manuscript records two incidents that justify the discipline. The same-length swap of Leg E was caught only by the oracle. In the other direction, an oracle constant was once a 70-digit truncation of a 76-digit decimal, and the circuit was right. The rule since: a mismatch is resolved from source, never by copying the circuit's output into the oracle.

The 1,101 bytes divide four ways. Fixed constants (magics, version bytes, the model digests, the beacon chain hash): they are enforced by the guest under the pinned key, and an independent verifier may decode the statement and compare them with `PINS_final.json` and Appendix A of the manuscript. Supplied session and context fields that the checks consume (the identifier, row count, depth, `S_0`, `S_N`, the manifest and chain-log digests). Values the relation recomputes or derives (the context digest, the session root, the typed root, the txid and header hash, the contentRoot and receipt digest, and the trapdoor flag, which is derived from the witness). And learned outputs (the coupling numerator, the pose verdict and logit sums). The two rounds are supplied values whose signatures the relation verified. Section 5.1's byte table is authoritative. The statement carries no pixels and no chain digests of the row itself; it carries the session root instead.

8. Example seven: the whole session in one proof

The per-row proof verifies two transitions of the chain. A second program (Section 9.5) verifies the chain itself. It reads the complete 712-row chain log (367,429 bytes), verifies one quicknet signature for each of the 66 distinct rounds the session consumed (the signature logged on the first row of each round; later rows' repeated signature fields are not re-verified), checks that every row's logged beacon value equals the verified value of its round, recomputes all 712 transitions with the same advance function (the last landing on S_N), requires the rounds never to decrease, and rebuilds the ordered-session tree of Example three exactly as the per-row relation defines it. Its 279-byte statement publishes the row count, depth, number of rounds, first and last rounds (31521605 and 31521705), S_0 , S_N , the log and manifest digests, the context digest, the root and the session identifier.

It executes in 368,844,955 instructions, of which 324 million are the 66 signature verifications, and was proved in 477.7 s on one A100. Its ELF and key were reproduced on the proving box and again on the development machine by a fail-closed build script.

Because its context digest and root equal the ones in every per-row statement, the 259 per-row proofs and the chain proof are known to describe one tree of one beacon-seeded log. What the chain proof does not establish, and the manuscript says so: the operator supplied the log. Nothing in it says the log is an authentic acquisition chronology, that its frame digests name frames a camera produced, or that its metadata and round schedule were committed anywhere before the session ran. It also does not re-hash the frames; that would cost about 500 billion instructions, and the per-row proofs do it for the rows they cover.

9. What the proofs establish, and what they do not

Established (Theorem 1, execution binding). If the verifier accepts a proof under the pinned key, then the published coupling numerator, pose verdict and logit sums are the frozen networks' outputs on the frame whose BLAKE3 is bound into the chain state and the session leaf; the published rounds are those whose signatures verified; and the published txid and header hash are those of the transaction bytes and the header reached by the Merkle branch. This rests on the zkVM's soundness (A1), the build pinning (A2) and the hash functions (A3), and on nothing else. In particular it does not depend on the neural networks being any good.

Conditional (Proposition 2). Under A1 to A4 and the unproved premise A8, and only when the predecessor record was fixed before round r_{t-1} 's scheduled release, the pattern the proved frame was checked against is the value A8 says was unavailable before that release. A party who chooses the predecessor record after seeing the signature is outside A8. Whether the *light in the frame* was emitted after that time additionally needs P1 (the frame is genuine sensor output) and P2 (for a threshold τ fixed before applying the corollary, a published numerator $q_t \geq \tau$ implies, on the relevant acquisition distribution, that the recorded illumination was this pattern rather than another pattern or none). The proof establishes neither. A live optical relay of a displayed scene satisfies both premises, so nothing here excludes relay or says where the scene was.

Not established. That the anchor bounds the record from above (the custodian held the trapdoor). That the pose verdict means a physical pose (a fixed function's output, confounded with time on the one take). That the chain log is authentic (operator-supplied). That rows 260 to 711 are anchored (they lie outside the 260-row prefix; covering them without exercising the disclosed chameleon-equivocation capability needs a new anchor transaction; row 0 is refused because it has no predecessor). The manuscript's Section 11 lists these; Section 8 gives the diagnostics with their small samples and confounds labelled as such.

The honest one-line summary is the manuscript's own: the proof establishes execution binding exactly; the physical reading is conditional and labelled; the models are diagnostics and labelled; the anchor is a receipt and labelled.

10. Verify it yourself

Everything below follows VERIFY.md v2.9 in the bundle and runs from the bundle's root directory.

```
sha256sum -c SHA256SUMS
(cd source/rust && sha256sum -c ../../final_relation/SOURCE_TREE_SHA256SUMS)
(cd source/rust/zeebeam_chain_spl_candidate && \
    sha256sum -c ../../../../final_relation/chain/CHAIN_SOURCE_SHA256SUMS)
(cd source && sha256sum -c ../final_relation/DEPENDENCY_SHA256SUMS)

(cd verifier/standalone_verifier && cargo build --release --locked)
ZB_VERIFY_BIN=verifier/standalone_verifier/target/release/zeebeam-standalone-verifier

"$ZB_VERIFY_BIN" \
    final_relation/ceremony2/row_096_groth16_proof.bin \
```

```

final_relation/ceremony2/row_096_groth16_public_values.bin \
0x0019d16cfe911d943315c04235c792fcccc3e0759aa3095a941b8508178e0490

"$ZB_VERIFY_BIN" \
final_relation/chain/chain_groth16_proof.bin \
final_relation/chain/chain_groth16_public_values.bin \
0x005402a848fdc787e32c0a110ca2662dd61c580481b4e418d5400a52fbd7df50

for p in all_rows/proofs/row_*_groth16_proof.bin; do
"$ZB_VERIFY_BIN" "$p" "${p%_proof.bin}_public_values.bin" \
0x0019d16cfe911d943315c04235c792fcccc3e0759aa3095a941b8508178e0490 |
grep -c VERIFIED
done | sort | uniq -c

python3 verifier/decode_statement.py \
final_relation/ceremony2/row_096_groth16_public_values.bin

```

Expected: the four digest ledgers pass (3,069, 38, 7 and 57 entries; the first covers every regular file in the bundle except itself); each verifier run ends VERIFIED after rejecting a flipped statement byte and a wrong key; the fleet loop prints 257 1; the decoder prints the fields of Section 5.1 as JSON, whose fixed fields you compare with Appendix A of the manuscript.

Then do the two things the proof cannot do for you. Confirm that the published header hash is that of canonical Zcash mainnet block 3456294 and that the transaction 8d1672...f206 is in it. Look up rounds r_{t-1} and r_t on quicknet and note their scheduled release times: under A8, the release of r_{t-1} is the claimed lower bound on when the row's pattern could have been computed, and it becomes a bound on recorded light only under P1 and P2; the row's own round r_t concerns the following frame.

Optional, with SP1 6.4.0 installed and the tree placed as source/README.md describes: run source/build_reproducible.sh and source/build_reproducible_chain.sh. Each exits 0 only if the rebuilt ELF's size, SHA-256 and key equal the pins.

11. The numbers in one place

quantity	value	source
session	712 rows, 301 s, 22 Aug 2026 03:09:45 to 03:14:46 UTC; 66 beacon rounds	manuscript §4
raw frame	5,320 × 4,600 BayerRG8, 24,472,000 B	§4

quantity	value	source
pattern	1,920 × 1,080 × 3, rendered from the chain state	\$4
per-row program	4,149,712,293 instructions (row 96); 4,146,753,922 to 4,154,781,545 over rows 1 to 259	\$6, \$7.6
per-row statement	1,101 B	\$5.1
raw Groth16 proof	356 B; framed SP1 object 2,791 to 2,795 B	\$7.1, \$7.2
verification key	0x0019d16c...0490; ELF 1,171,416 B, sha256 8bcadf53...11db	Appendix A
proving, per row	12 min 08 s to 15 min 56 s on one A100 across the fleet, median 12 min 36 s	\$7.1, FLEET_STATS.json
proofs made	259 anchored rows (2 in ceremony 2, 257 on an eight-A100 fleet); all standalone-verified	\$7.1, \$7.6
fleet cost	4,073 instance-minutes, about \$135 at \$1.99 per hour	\$7.1
chain program	368,844,955 instructions; 279 B statement; proved in 477.7 s; key 0x005402a8...df50	\$9.5
verification	1.64 s and 2.3 MB for one proof plus the two negative checks; 917,512-byte verifier; 209 locked packages	verifier/results/halo_timing_20260903.txt
anchor	txid 8d1672...f206, block 3456294, prefix rows 0 to 259; trapdoor held by the custodian	\$4, \$9.4
referee passes	nineteen (nine rounds before v3.0; a tenth verification pass on the additions; an eleventh publication-byte pass; a twelfth verification pass on the applied text; seven closing passes)	\$7.4

Log

- 1.16 (2026-09-07, BOSUN): manuscript v3.20 (*ZeeBeam: The Zero-Knowledge Beam*); the companion_to line and the introduction follow; no other change.
- 1.15 (2026-09-06, BOSUN): manuscript v3.16; PDF re-rendered with the long-token fix; the 1.13 entry's manuscript label restored to v3.14 (history). No other change.
- 1.14 (2026-09-06, BOSUN): manuscript v3.15 (referee trail published again); no other change.
- 1.13 (2026-09-06, BOSUN): bundle ledger count 3,069 after the bundle repair; manuscript v3.14. No other change.
- 1.12 (2026-09-05, BOSUN): manuscript v3.13; nineteen passes. No other change.
- 1.11 (2026-09-05, BOSUN): manuscript v3.12; eighteen passes. No other change.
- 1.10 (2026-09-05, BOSUN): manuscript v3.11; seventeen passes. No other change.
- 1.9 (2026-09-05, BOSUN): status line and manuscript references (v3.10) brought into agreement; sixteen passes. No other change.
- 1.8 (2026-09-05, BOSUN): fifteen passes; manuscript v3.9. No other change.
- 1.7 (2026-09-05, BOSUN): VERIFY 2.9; fourteen passes; manuscript v3.8. No other change.
- 1.6 (2026-09-05, BOSUN): Sol's closing pass applied: chronology attributed to the project records; P2 stated with a threshold fixed in advance; VERIFY version and the ledger count; thirteen passes; manuscript v3.7.
- 1.5 (2026-09-05, BOSUN): Sol's verification pass applied: witness as the thirteen guest inputs; memo binding stated exactly; fixed fields enforced by the guest, comparable by the reader against the pins; VERIFY 2.7 and the ledger counts; twelve passes; manuscript v3.6.
- 1.4 (2026-09-05, BOSUN): Sol's eleventh pass applied: hash hiding stated as the manuscript states it (binding under A3, no hiding guarantee); verification time quoted from the bundled record; new-anchor sentence qualified by the disclosed equivocation capability; manuscript reference v3.5; companion_to points at the public tree.
- 1.3 (2026-09-05, BOSUN): named ZeeBeam with the manuscript (v3.4); Dark Lantern is the research programme. No technical content changed.
- 1.2 (2026-09-05, BOSUN): renamed with the manuscript (Dark Lantern the project, ZeeBeam the proved recording); manuscript reference v3.3. No technical content changed.
- 1.1 (2026-09-03, BOSUN): Sol ultra audit applied. Privacy wording narrowed to the manuscript's (execution binding; no pixels in the statement; no privacy

theorem claimed); hash hiding caveat; toy Schnorr moved to a one-bit challenge with the composite-order counterexample; Fiat-Shamir as a quantified soundness error; the legs are pairing and Baby Jubjub checks, not Schnorr; Merkle path wording; leaf and context fields completed; key derivation and A2; exact fleet time range; verifier measurements now cite the bundled record; witness list per the manuscript; pairing syscall; nominal pattern; XOF separated from the render; typed root belongs to the coupling pair; pose reduction; both out-of-band checks; memo argument via A5; anchor leg rewritten as receipt plus demonstrated opening with the objects defined; statement taxonomy; both transitions; chain signature check stated exactly; Proposition 2 with A8's scope; runnable verification commands; ledger counts.

- 1.0 (2026-09-03, BOSUN): written on the principal's request for a walk from the simplest example to the manuscript's relations. Toy values computed on the development machine; real-system figures from the manuscript and the bundle. Nothing here is published.